

TraceX: Autonomous Distributed Root Cause Analysis

Real-Time Distributed Telemetry Simulation, Dual-Tier Anomaly Detection, Graph-Based MicroRCA Diagnostics, and Gemini AI Remediation with Interactive Storefront Validation.

System Version: 2.0.0
Enterprise

Stack: Next.js 16 (App Router) + FastAPI +
Google Gemini

Target Domain: Distributed Microservices & SRE
Automation

1. Executive Summary & Problem Formulation

Problem Statement

In modern microservice architectures (e.g., Netflix, Uber, Amazon), applications run across hundreds of interconnected services communicating asynchronously. When an infrastructure component degrades (e.g., database connection pool exhaustion), failures rapidly cascade upstream across APIs and gateways.

⚠️ The SRE Alert Fatigue Crisis

Traditional monitoring tools generate alert storms across dozens of symptom nodes. Engineers spend 70% of MTTR (Mean Time to Resolution) identifying *which service failed first* vs. which services are merely downstream victims.

💡 The TraceX Solution

TraceX autonomous engine continuously observes telemetry, maps service dependency DAGs, eliminates symptom nodes using span self-time attribution, and produces LLM-synthesized runbooks within **under 3 seconds**.

2. Simulated Microservice Topology

Architecture

TraceX runs an active 8-microservice digital twin simulating typical cloud-native e-commerce dependencies:

```
[ Edge Ingress ] CDN (Edge caching layer, 5ms SLA) | [ Ingress Proxy ] API Gateway (Rate-limiting & Routing, 15ms)
└──────────────────┘ └──────────────────┘
┌──────────────────┘ ┌──────────────────┘ [ Core Services ] Auth Service Order Service Inventory Service (OOM/GC) (JWT/Tokens) | |
└──────────────────┘ └──────────────────┘ └──────────────────┘ └──────────────────┘
┌──────────────────┘ ┌──────────────────┘ [ Downstream ] Payment Notif Postgres DB ←──┐ Service Service (Connection pool, 8ms)
```

3. End-to-End System Working in 8 Clear Steps

Core Engine

Step 1: Real-Time Telemetry Synthesis & Digital Twin Heartbeat

PHASE 1

The backend maintains an in-memory **Digital Twin** representing 8 interconnected microservices. Every **1.0 second**, the continuous simulation worker ticks:

- Generates live traffic metrics: Latency percentiles (P50, P95, P99), error rates, throughput (RPS), and CPU/memory utilization.
- Synthesizes distributed tracing spans adhering to the OpenTelemetry schema (UUID trace IDs, parent-child span IDs, protocols, durations).
- Feeds all data into a thread-safe sliding window ring buffer (retaining 60s baseline history).

Step 2: Dual-Tier Anomaly Detection Engine

PHASE 2

To prevent false positives while rapidly catching abrupt failures, TraceX implements a dual-tier anomaly detector:

Tier 1 Statistical Z-Score: $Z = (X - \mu) / \sigma$ (Evaluated over rolling 60-second window, threshold $Z \geq 3.0$)

Tier 2 Absolute Guardrails: Error Rate $\geq 10.0\%$ OR Latency $\geq 500\text{ms}$ OR RPS drop $\geq 50\%$

This dual approach ensures both subtle slow drifts and catastrophic step-function crashes are immediately captured.

Step 3: Chaos Injection & Cascading Failure Propagation

PHASE 3

Fault scenarios can be injected from the SRE dashboard or triggered directly by user food orders:

Scenario	Root Target	Cascading Timeline & Impact
DB Overload	postgres-db	T+0s: DB pool saturates (P99 > 4200ms) → T+8s: Order Service threads block → T+15s: Gateway returns 504.
Auth Crash	auth-service	Process terminates. Token verification fails immediately → Ingress Gateway rejects all traffic (HTTP 401/502).
Payment Partition	payment-service	Outbound payment socket timeout → Order Service circuit breaker trips → Checkouts halted.
Memory Leak	inventory-service	Heap reaches 95% → Stop-The-World GC thrashing spikes latency to 3500ms → Stock reservation stalled.
CDN Latency Spike	cdn	Edge routing degradation spikes ingress latency to 850ms → System-wide client slowdown.

Step 4: MicroRCA Root Cause Localization Algorithm

PHASE 4

When multiple services report anomalies simultaneously, identifying the root cause vs. collateral symptoms is critical:

Algorithm MicroRCA-Localization(TelemetryBatch, AnomalyEvents, DependencyDAG): 1. Construct Subgraph of currently anomalous nodes and incoming/outgoing edges. 2. For each anomalous service S: Calculate Self-Time = Total_Duration(S) - $\sum$ Child_Call_Durations(S.children) 3. Prune symptom nodes: If S has high latency but Self-Time < 15%, S is merely waiting on dependencies. 4. Trace directed causal edges to find the leaf culprit with highest anomalous self-time bottleneck. 5. Compute Confidence Score: Normalized harmonic mean of Z-score deviation and anomaly centrality. 6. Return: Root Cause Service ID, Primary Fault Metric, and Confidence Percentage (e.g. 98.5%).

Step 5: Failure Propagation Cascade & Blast Radius Estimation

PHASE 5

Once the root cause is localized, TraceX executes graph traversal forward to construct the complete incident timeline:

- Propagation Pathway:** Maps the chain of failure (e.g., postgres-db → order-service → api-gateway) with inter-hop latency delays.
- Blast Radius Estimation:** Analyzes dependency centrality to calculate the exact number of impacted services (e.g., 3/8) and estimated affected active user sessions (e.g., ~4,300 users).

Step 6: Generative AI Incident Explanation & Runbook Synthesis

PHASE 6

TraceX leverages **Google Gemini (gemini-1.5-flash / gemini-2.0-flash)** via an async worker thread:

- **Prompt Context:** Feeds the mathematical MicroRCA output (root cause, confidence %, P99 anomaly values, propagation chain, blast radius).
- **Structured JSON Output:** Enforces 3 distinct fields:
 1. **summary:** Executive 2-3 sentence technical overview.
 2. **causal_chain:** Step-by-step narrative of the failure cascade.
 3. **suggested_fix:** Concrete remediation steps (pool sizing, circuit breaker thresholds, caching).
- **Deterministic Fallback:** If no API key is provided, an offline rule-based knowledge engine generates accurate explanations without failing.

Step 7: Real-Time SRE Visual Dashboard Synchronization

PHASE 7

The diagnosis is broadcasted over **Socket.IO** to the Next.js React Flow dashboard:

- **Dynamic Topology Graph:** The root cause node pulses with an active **red glow ring**; affected downstream nodes pulse with **amber warning borders**; communication edges turn red.
- **Sparkline Metrics:** Live P95 latency and error rate charts update every 1 second.
- **Chronological Incident Timeline:** Displays timestamped entries with colored severity badges.

Step 8: Interactive Food App Demonstration & 1-Click Auto-Healing

PHASE 8

To prove real-world usability, TraceX includes **TraceBites** (a complete gourmet food delivery app at /store):

- **Simulated Bug Checkout:** Users place orders with simulated faults. The transaction fails with HTTP 504 and renders a customer alert plus a technical **TraceX Root Cause Diagnostic Card**.
- **1-Click Auto-Heal & Normal Order:** Clicking "*Auto-Heal & Retry*" or selecting "*Normal Order (200 OK)*" transmits an `autoHeal: true` payload. The digital twin clears all active faults, restores green baseline health, and immediately issues **Order Confirmed**.

4. Technical Specifications & Stack

Specs

Layer	Technologies	Key Responsibilities
Frontend	Next.js 16 (App Router), Turbopack, Tailwind CSS, Lucide React	Responsive SRE dashboard, TraceBites food store, state synchronization.
Visualization	@xyflow/react (React Flow), Dagre Layout, Recharts	Hierarchical DAG layout, dynamic node borders, real-time sparklines.
Backend API	Python 3.13, FastAPI, Uvicorn, Pydantic v2	REST endpoints, async workers, telemetry ingestion, thread offloading.
Streaming	python-socketio (ASGI mode), socket.io-client	Bi-directional real-time telemetry streaming and chaos event broadcast.
AI & Analytics	Google Gemini API (1.5 Flash / 2.0 Flash), NumPy, scikit-learn	Statistical Z-scores, span self-time attribution, natural language runbooks.